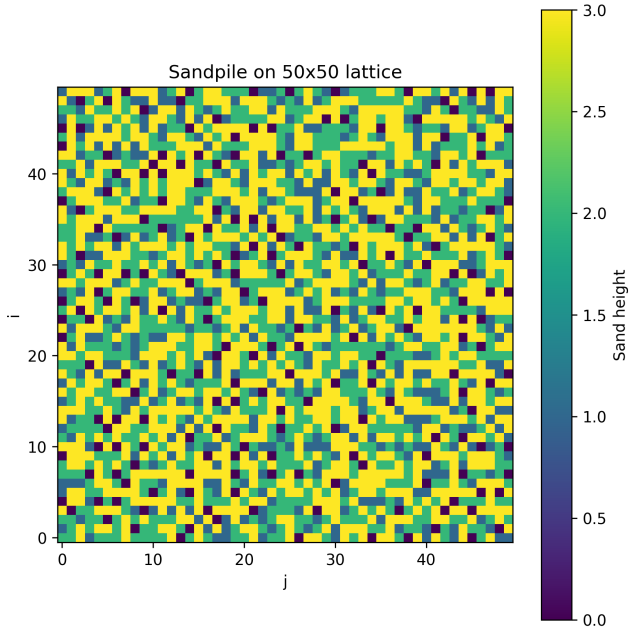
THE UNIVERSITY OF MELBOURNE

Final Report

The Art of Scientific Computing: Self-Organized Criticality in the Bak–Tang–Wiesenfeld Sandpile Model

Author:
Kaiyu Wu

Subject Handbook code:
COMP-90072



Faculty of Science
The University of Melbourne

29th May 2026

Subject co-ordinators: A/Prof. Roger Rassool & Prof. Harry Quiney

Abstract

This report investigates self-organized criticality in the Bak–Tang–Wiesenfeld sandpile model. The model is simulated on a finite two-dimensional lattice, where grains are added slowly and unstable sites topple by distributing grains to their nearest neighbours. Open boundaries allow grains to leave the system, making the model a driven dissipative system. The main observables are the average pile height, avalanche size, avalanche area, avalanche loss, and avalanche length. The average height is used to test whether the system reaches a statistically stationary state, while avalanche distributions are used to test for scale-invariant behaviour. The standard model with uniformly random grain addition is compared with an extended model in which grains are added preferentially near the centre of the lattice. The aim is to determine whether both models display evidence of self-organized criticality and to understand how changing the driving rule affects the avalanche statistics.

Contents

Chapter 1

Formatting options

The best way to view this document is in a split-view format, with the `.tex` file open and the corresponding portion of the `.pdf` output also visible.

1.1 Equations and tables

You can write an equation without a label by using an asterisk:

$$f(x) = (x + a)(x + b).$$

Alternatively, you can number and label your mathematical expressions:

$$\mathcal{L}_{\mathrm{EM}} = \frac{1}{2}\left(\epsilon_0 \left|\boldsymbol{E}\right|^2 + \frac{1}{\mu_0}\left|\boldsymbol{B}\right|^2\right) - \phi\varrho_{\mathrm{free}} + \boldsymbol{A}\cdot\boldsymbol{J}_{\mathrm{free}} + \boldsymbol{E}\cdot\boldsymbol{P} + \boldsymbol{B}\cdot\boldsymbol{M}. \tag{1.1}$$

This way, we can later reference something we wrote earlier, like the electromagnetic Lagrangian density in non-relativistic vector notation shown in Equation (1.1).

The equation environment allows you to construct a number of objects, such as arrays and algined sets of equations. For instance, we might want to write a general matrix:

$$\boldsymbol{c} = \begin{pmatrix} c_{11} & c_{12} & c_{13} & \dots & c_{1n} \\ c_{21} & c_{22} & c_{23} & \dots & c_{2n} \\ c_{31} & c_{32} & c_{33} & \dots & c_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_{m1} & c_{m2} & c_{m3} & \dots & c_{mn} \end{pmatrix}. \tag{1.2}$$

We might alternatively want to write a set of simultaneous equations. If future descriptions require for all of them to have unique labels, try using the 'subequations' environment as well as the 'eqnarray' environment.

$$\begin{aligned} 3x + 5y + z &= 5 && \text{(1.3a)} \\ 7x - 2y + 4z &= 3 && \text{(1.3b)} \\ -6x + 3y + 2z &= 1. && \text{(1.3c)} \end{aligned}$$

This will allow you to talk about the set of simultaneous equations as a whole by referring the reader to Equation (1.3), or else to a specific statement like Equation (1.3b).

Sometimes you'll want to derive a result, which in its clearest form requires more than one line of algebra. There are a few ways to accomplish this, and the best choice will depend on the task at hand and your personal taste. You can instruct the compiler *not* to give a label to one or more of the lines as you go.

$$\begin{aligned} & R_x(\varepsilon)R_y(\varepsilon) - R_y(\varepsilon)R_x(\varepsilon) \\ &= \left(1 - \frac{iJ_x\varepsilon}{\hbar} - \frac{J_x^2\varepsilon^2}{2\hbar^2}\right)\left(1 - \frac{iJ_y\varepsilon}{\hbar} - \frac{J_y^2\varepsilon^2}{2\hbar^2}\right) - \left(1 - \frac{iJ_y\varepsilon}{\hbar} - \frac{J_y^2\varepsilon^2}{2\hbar^2}\right)\left(1 - \frac{iJ_x\varepsilon}{\hbar} - \frac{J_x^2\varepsilon^2}{2\hbar^2}\right) \\ &= \left(1 - \frac{iJ_z\varepsilon^2}{\hbar}\right) - 1 + \mathcal{O}(\varepsilon^3). \end{aligned} \tag{1.4}$$

A table of data is sometimes necessary, for which there is a dedicated environment. An example of this is given in Table (1.1) – note that the counter for tables is separate from the counter for equations. If this isn't to your liking, you can change the enumeration style of one or both. (Google has plenty of suggestions).

description		value		
electronic charge	e	$1.6021766 \times 10^{-19}$C		
proton mass	m_p	$1.6726218 \times 10^{-27}$kg	or	$938.27203\text{MeV}/c^2$
electron mass	m_e	$9.1093822 \times 10^{-31}$kg	or	$510.99891\text{keV}/c^2$
permittivity of vacuum	ϵ_0	$8.8541878 \times 10^{-12}$F/m		
permeability of vacuum	μ_0	1.2566371×10^{-6} H/m	or	$4\pi \times 10^{-7}$H/m
speed of light in vacuum	c	$2.9979246 \times 10^{+8}$ m/s		
electrostatic constant	k_e	$8.9875518 \times 10^{+9}$ m/F	or	$1/4\pi\epsilon_0$
Planck constant	h	$6.6260696 \times 10^{-34}$ J$\cdot$s	or	$4.1356675 \times 10^{-15}eV\cdot$s
reduced Planck constant	$\hbar$	$1.0545717 \times 10^{-34}$ J$\cdot$s	or	$6.5821193 \times 10^{-16}eV\cdot$s

Table 1.1: A table of measured constants.

1.2 Pictures and figures

Try to stick to a few data formats if possible. Vector and pdf data types are ideal, because they minimise the overall size of the document and you can zoom/print to any quality without pixelisation.

You can write a program that returns data or a visual representation of it (graph, coloured array, surface plot etc.), and in the latter case, you can take the resulting image and copy to your 'figures' folder (this keeps the internal address conventions neat and tidy) and link the document directly to your report, like so:

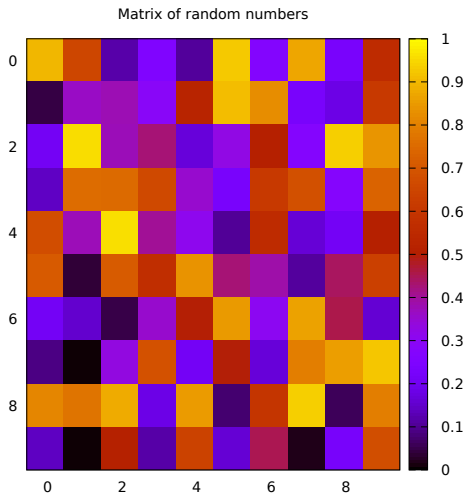


Figure 1.1: A random square matrix, generated by a `Fortran` program.

If you need to provide a diagram of your own creation, there are several options available. You can feel free to choose whichever method your experience has endowed you with, or else represents the best compromise between effort spent and professional appearance of the final product. For instance, you could hand-draw your diagram, scan or take a photo of it, upload the resulting image to the correct directory, crop/filter it and add to your document, as shown in the LHS of Figure (1.2).

Alternatively, you could use some available computer software to get the job done. Here we will give you an example made in Inkscape, because it is an open-source package which is available for most operating systems. Here you will make your diagram (there are plenty of tutorials and examples available online), adjust the document borders to your satisfaction, save a vector-type copy for later editing purposes (for instance, '`diagram.svg`', and then also save a pdf version ('`diagram.pdf`'). It is the second file-type which you can link to your LaTeXdocument with the usual features. The comparison between a hand-drawn and computer-rendered image is fairly stark – see the RHS of Figure (1.2).

If you wish to take the last example further, try exploring some of Inkscape's features. It allows you to plot functions, write mathematical expressions (although this option does not always work) and vectorise/edit a surprisingly large assortment of existing images. If you would like your final document to look seamlessly

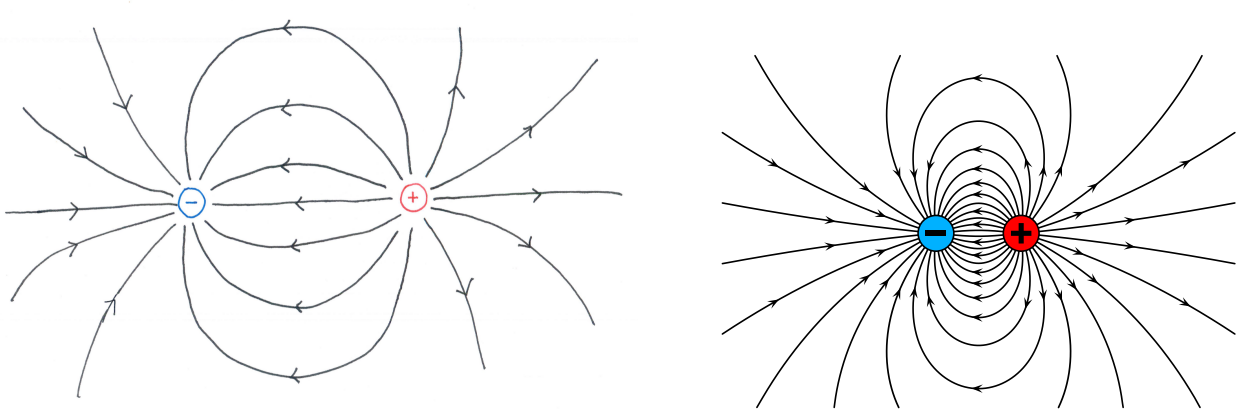


Figure 1.2: A sketch of some electric field lines around an ideal electric dipole.

consistent, you can even make diagrams with text and equations of the same formatting. (This involves writing everything in a dummy LaTeXdocument, exporting and vectorising it online, importing that vector document into Inkscape, playing around with character size compared to final drawing size in the document, and then exporting that diagram as a pdf to be placed back into LaTeX... possibly not a worthwhile pursuit for the purposes of this subject.)

1.3 Computer code

1.3.1 Code input/output

Talk about the language that you used, maybe referencing an integral in [1] or a numerical algorithm in [4] if you used `Fortran`.

1.3.2 Code written verbatim

This can be done with the `listings` and `verbatim` packages. You should modify the preamble document so that these packages correctly interpret the programming language that you used.

```
      FUNCTION PINEWT(N)
      IMPLICIT DOUBLE PRECISION (A-H,O-Z)
C*********************************************************************C
C     PINEWT EVALUATES THE INFINITE SERIES REPRESENTATION             C
C     FOR PI AS A TRUNCATED SUM.                                      C
C*********************************************************************C
C
C     ENSURE N IS A LEGAL VALUE
      IF(N.LT.0) THEN
        WRITE(6,*) 'In␣PINEWT:␣illegal␣value␣for␣N.',N
        RETURN
      ELSEIF(N.EQ.0) THEN
        PINEWT = 2.0D0
      ENDIF
C
C     INITIALISE SOME COUNTERS
      TWOPOW = 1.0D0
      FACKNM = 1.0D0
      FACKDM = 1.0D0
      PINEWT = TWOPOW*FACKNM*FACKNM/FACKDM
C
C     LOOP OVER SERIES ORDER N TIMES, ADDING EACH TERM TO TOTAL
      DO I=1,N
        TWOPOW = 2.0D0*TWOPOW
        FACKNM = DFLOAT(I)*FACKNM
        FACKDM = DFLOAT(2*I+1)*DFLOAT(2*I)*FACKDM
        PINEWT = PINEWT + TWOPOW*FACKNM*FACKNM/FACKDM
      ENDDO
C
C     DOUBLE THE RESULT (SEE WIKIPEDIA FORMULA)
      PINEWT = 2.0D0*PINEWT
C
      RETURN
      END
```

Chapter 2

Important definition for report

2.1 SOC meaning

Self-organized criticality was introduced by Bak, Tang, and Wiesenfeld [2, 3], usually abbreviated as SOC, describes a class of complex systems that naturally evolve towards a critical state without the need for carefully tuned external parameters. In such systems, small local changes can sometimes produce responses of many different sizes, ranging from negligible events to large-scale avalanches. This means that the size of an event is not necessarily proportional to the size of its cause.

A standard example of SOC is the Bak–Tang–Wiesenfeld sandpile model. In this model, grains of sand are added slowly to a lattice. When the height at a site exceeds a fixed threshold, that site topples and redistributes grains to its neighbours. This redistribution may cause further topplings, producing an avalanche. Although the local rules are simple, the collective behaviour of the system can be highly non-trivial.

A key feature of SOC systems is scale invariance. After the system reaches a statistically stationary state, there is no single characteristic avalanche size. Instead, small avalanches occur frequently, while large avalanches occur more rarely, often following a power-law distribution. This makes the sandpile model a useful framework for studying how complex macroscopic behaviour can arise from simple microscopic rules.

In this chapter, we introduce the idea of SOC and use the sandpile model as a simple computational example. The goal is to investigate how repeated local interactions can drive the system towards a critical state, and how avalanche statistics can be used to identify scale-invariant behaviour.

2.2 BTW-model Notation

Let the sandpile be defined on an $N \times M$ rectangular lattice. Write

$$h_t(i, j)$$

for the number of grains at site (i, j) after the t-th grain addition, where

$$1 \le i \le N, \qquad 1 \le j \le M.$$

A site (i, j) is called *unstable* if

$$h_t(i, j) \ge 4.$$

When an unstable site topples, its height decreases by 4, and each of its four nearest neighbours receives one grain:

$$h(i, j) \mapsto h(i, j) - 4,$$

$$h(i \pm 1, j) \mapsto h(i \pm 1, j) + 1, \qquad h(i, j \pm 1) \mapsto h(i, j \pm 1) + 1.$$

If a grain is sent outside the boundary of the lattice, then it is lost from the system.

Question 1

Why is it that a system exhibiting scale invariance will have phenomena which are described by a power law?

Let s denote the size of an event. For example, in the sandpile model, s may denote the number of topplings in an avalanche. Let $P(s)$ denote the frequency or probability distribution of events of size s.

Scale invariance means that if we rescale the event size by a positive constant factor $a > 0$, the distribution should have the same shape, up to an overall multiplicative factor. That is, we expect

$$P(as) = C(a)P(s),$$

where $C(a)$ depends only on the scaling factor a, not on s.

A function satisfying this kind of scaling relation is a power law. Suppose

$$P(s) = Cs^{-\alpha},$$

where $C > 0$ and $\alpha > 0$ are constants. Then

$$P(as) = C(as)^{-\alpha} = Ca^{-\alpha}s^{-\alpha} = a^{-\alpha}P(s).$$

Thus, after rescaling $s \mapsto as$, the distribution is changed only by the multiplicative constant $a^{-\alpha}$, while the functional form is unchanged.

Therefore, if a system has no characteristic event-size scale, then the natural form of its event-size distribution is

$$P(s) \propto s^{-\alpha}.$$

This explains why scale-invariant systems are commonly associated with power-law distributions. In the BTW sandpile model, this means that avalanche sizes may occur over a wide range of scales, with small avalanches being common and large avalanches being rare, but not exponentially rare, this will be seen from the data I collected.

Question 2

In what ways does the BTW model satisfy the properties required for a system to exhibit self-organized criticality?

A system exhibiting self-organized criticality usually has the following properties:

(i) it is slowly driven;

(ii) it has local interactions;

(iii) it has dissipation(we lose the quantity that is been slowly added);

(iv) it evolves toward a critical state without fine-tuning of external parameters;

(v) it produces avalanches over many different scales.

The BTW model satisfies these properties as follows.

Slow driving

At each time step, only one grain of sand is added to the lattice. Hence the system is driven slowly and locally. The system is not forced by a large global perturbation *. Instead, small local changes gradually build up stress in the system, causing avalanches.

Local interactions

The toppling is local. When a site (i, j) topples, it only affects its four nearest neighbours:

$$(i+1, j), \quad (i-1, j), \quad (i, j+1), \quad (i, j-1).$$

There are no long-range interactions in the basic model. Therefore, all large-scale behaviour comes from repeated local redistribution of sand.

Dissipation

The finite boundary of the lattice provides dissipation. If a boundary site topples, then some grains may be sent outside the lattice and are deleted from the system. Thus sand can enter the system through grain addition and leave the system through the boundary.

*It should be said that this is within the realm of what uuually happens and we don't consider largeperturbations to the syssem (succ as a large meteorite crashing on Earth whicc may ccange the tecconicsdrassically, cause enormous foress ffres, result in epidemicsas the ecosyssem ccanges,and cause the ffnancialmarket to crash as civilization collapses)

Self-organization

No external parameter needs to be tuned to a critical value. Starting from an empty lattice, the average amount of sand initially increases. Eventually, avalanches carry sand to the boundary, where grains are lost. After a transient period, the system reaches a statistically stationary regime where the average input of sand is balanced by the average loss of sand.

Avalanches over many scales

A single added grain may cause no toppling, a small avalanche, or a very large avalanche. Hence the system produces events over a broad range of sizes. In the critical regime, one expects avalanche statistics to be broad and often approximately described by power laws.

Therefore, the BTW model is a standard example of a self-organized critical system because it is slowly driven, locally interacting, dissipative, and naturally evolves toward a critical state without external fine-tuning.

Question 3

Why is it important for the table to have finite boundaries? What if we had an infinite lattice, but dropped the sand on only a finite subset of the lattice?

The finite boundary is important because it allows sand to leave the system. This is the dissipation mechanism of the BTW model.

Define the total amount of sand in the system by

$$S_t = \sum_{i=1}^{N} \sum_{j=1}^{M} h_t(i,j).$$

Each grain addition increases S_t by 1. Interior topplings conserve the total amount of sand, because the toppling site loses 4 grains and its four neighbours receive 1 grain each. Hence the total change from an interior toppling is

$$-4 + 1 + 1 + 1 + 1 = 0.$$

However, boundary topplings can decrease S_t, because some grains are sent outside the lattice and are deleted. Therefore, in a long-time statistically stationary state, we expect the average input rate to balance the average dissipation rate:

$$\text{average sand input rate} = \text{average sand loss rate}.$$

Since one grain is added per time step, the average loss rate should also approach one grain per time step in the stationary regime.

If the lattice were infinite, there would be no boundary where sand could be lost. If sand were dropped only on a finite subset of an infinite lattice, then the local region might still generate avalanches, but the total amount of sand in the infinite system would keep increasing because no grains are deleted. Sand could spread farther and farther away, but it would not leave the system.

Thus, an infinite lattice lacks the same dissipative mechanism. Without dissipation, the model is no longer the same open driven-dissipative system. Consequently, it is not clear that it would reach the same kind of statistically stationary SOC state as the finite-boundary BTW model.

Hence finite boundaries are important because they make the BTW model an open system:

$$\text{sand enters by driving} \quad \text{and} \quad \text{sand leaves through the boundary}.$$

Question 4

Without simulating anything, what might you expect the average height to be once the system has reached a steady state? Compare your initial expectation to what your simulation does.

The average height is defined by

$$\langle h_t \rangle := \frac{1}{NM} \sum_{i=1}^{N} \sum_{j=1}^{M} h_t(i,j).$$

A stable site must satisfy

$$h_t(i,j) < 4.$$

Therefore, the possible stable heights are

$$h_t(i,j) \in \{0,1,2,3\}.$$

A first naive expectation is that the stable heights $0, 1, 2, 3$ might occur equally often. If this were true, then the average height would be

$$\langle h \rangle_{\text{naive}} = \frac{0+1+2+3}{4} = \frac{3}{2}.$$

So a very simple first guess is

$$\langle h \rangle \approx 1.5.$$

However, this assumption is too naive. The recurrent configurations of the BTW model are not uniformly distributed over all stable configurations. The system tends to organize itself near a critical state, where many sites are close to becoming unstable. Therefore, heights 2 and 3 should occur more frequently than a uniform distribution would suggest.

Thus a more realistic expectation is

$$1.5 < \langle h \rangle < 4.$$

For the standard two-dimensional BTW model with open boundaries and a large lattice, one typically expects the long-time average height to be around

$$\langle h \rangle \approx 2.1\text{–}2.2.$$

In simulation, one should expect the following behaviour. Starting from an empty lattice,

$$\langle h_0 \rangle = 0.$$

As grains are added, $\langle h_t \rangle$ initially increases. After sufficiently many grain additions, boundary losses begin to balance input. Then $\langle h_t \rangle$ should not become exactly constant, but should fluctuate around a stable long-time mean:

$$\langle h_t \rangle \approx \text{constant in a long-time statistical sense.}$$

Therefore, the system should reach a statistically stationary average height rather than a perfectly fixed height configuration.

Question 5

Explain the difference between a steady state, a statistically stationary state, and an equilibrium state. Do you expect the BTW model to reach an equilibrium state or a statistically stationary state? In terms of system observables, how might we characterize this state?

Steady state

A steady state means that the state of the system no longer changes in time. For the sandpile, a strict steady state would mean

$$h_{t+1}(i,j) = h_t(i,j)$$

for every site (i,j) and for all sufficiently large t.

The BTW model does not reach this kind of strict steady state because grains keep being added and avalanches keep changing the configuration.

Statistically stationary state

A statistically stationary state means that the microscopic configuration continues to change, but the statistical properties of the system become time-independent.

For example, after a transient period, the following observables may fluctuate around stable long-time values:

$$\langle h_t \rangle,$$

$$P(s),$$

where $P(s)$ is the avalanche-size distribution, and

$$\text{average sand loss rate.}$$

Thus the exact configuration h_t keeps changing, but its statistical behaviour becomes stable.

Equilibrium state

An equilibrium state usually refers to a closed system that has relaxed to thermodynamic equilibrium. In equilibrium, there is typically no net flux of mass or energy through the system, and the system often satisfies detailed balance.

The BTW model is not an equilibrium system. It is continuously driven by grain addition and continuously dissipates sand through the boundary. Therefore, it has a nonzero flux:

$$\text{input by grain addition} \longrightarrow \text{redistribution by avalanches} \longrightarrow \text{loss through boundary}.$$

Hence the BTW model is a non-equilibrium driven-dissipative system.

Expected state of the BTW model

The BTW model is expected to reach a statistically stationary state, not an equilibrium state. In this state, the microscopic configuration keeps changing, but long-time statistical observables stabilize.

We may characterize this statistically stationary state using observables such as:

$$\langle h_t \rangle = \frac{1}{NM} \sum_{i=1}^{N} \sum_{j=1}^{M} h_t(i,j),$$

$$P(s),$$

where s is the number of topplings in an avalanche,

$$P(A),$$

where A is the number of distinct toppled sites,

$$P(L),$$

where L is the amount of sand lost at the boundary, and also a chosen avalanche length observable.

In the statistically stationary SOC regime, we expect the average height to fluctuate around a long-time mean, while avalanche observables have stable long-time distributions. In particular, the avalanche-size distribution may approximately satisfy a power-law relation of the form

$$P(s) \propto s^{-\alpha}$$

for some exponent $\alpha > 0$ over an appropriate scaling range.

Appendix A

Program flowchart

A flowchart is a conceptual aid which highlights input and output that a program should have, as well as the consequences of any decisions that it must make. Sometimes, these will include *pseudocode*, which mimics some of the more common coding structures in a universal way. (For instance, you might say 'loop over all array addresses'.)

This particular flowchart indicates a program that reads three numbers from the terminal, and then produces the largest of the numbers back to the terminal. The colour scheme is not mandatory, but helps the reader to know what is going on at a glance.

This flowchart was drawn with the free 'Lucidcharts' extension to Google Docs, from which the result was exported as an `svg` file, and any unnecessary features were then cropped out with Inkscape. Choose any method you wish to create your own flowchart!

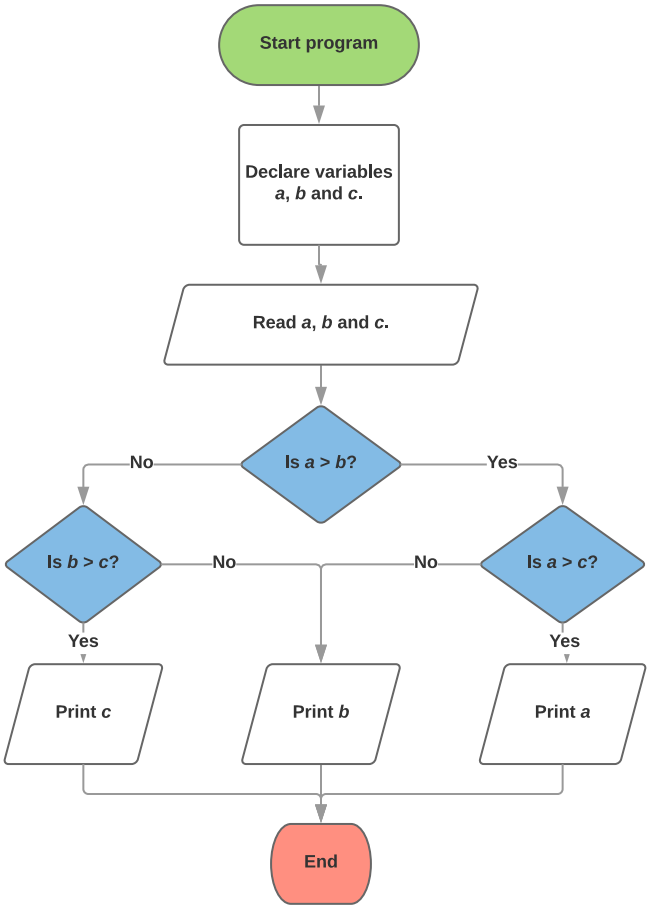


Figure A.1: A flowchart for the determination of the largest of three user-selected numbers.

Bibliography

[1] M. Abramowitz and I. Stegun. *Handbook of Mathematical Functions*. Dover Publishing Inc. New York, 1970.

[2] P. Bak, C. Tang, and K. Wiesenfeld. Self-organized criticality: An explanation of the 1/f noise. *Physical Review Letters*, 59(4):381–384, 1987.

[3] P. Bak, C. Tang, and K. Wiesenfeld. Self-organized criticality. *Physical Review A*, 38(1):364–374, 1988.

[4] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery. *Numerical recipes in Fortran (Cambridge)*. Cambridge Univ. Press, 1992.